

Final Report

ATM 기계 시뮬레이터

과 목	임베디드시스템S/W설계
담당교수	김태석 교수님
학 과	컴퓨터공학과
학 번	2019202084
성 명	배정우
날 짜	2025. 06. 01 (일요일)

1. 요약

Kernel function 수	사용한 Task의 수	Semaphore 예제 수	Message Queue 예제 수
1	6	2	4

<Kernel function>

커널 함수의 이름: OSResetDailyLimits()

위치: OS_CORE.C

이 커널 함수는 모든 계좌(AccountInfo)의 daily_used 필드를 0으로 초기화하여, 하루동안 누적된 출금 사용량을 초기화하는 역할을 한다. ATM 시스템에서 일일 출금 한도를 관리하는데 필수적인 기능이며, 이 함수는 별도의 ResetTask에서 주기적으로 호출된다. 계좌 정보 접근 시 데이터 무결성을 보장하기 위해 AccountSem 세마포어를 사용하여 동기화 처리를 수행한다.

<사용한 Task>

사용한 task의 수는 총 6개로, CardReaderTask, AuthTask, TransactionTask, CashDispenserTask, LogTask, ResetTask가 있다. 먼저 CardReaderTask는 (랜덤으로)카드 삽입 시, 카드 정보를 읽고 그 카드의 유효성을 검사한다. AuthTask는 CardReaderTask로부터 받은 카드의 정보로 계좌 탐색 후 PIN 입력 3회까지 인증을 시도한다. TransactionTask는 AuthTask로부터 받은 인증된 계좌에 대해 거래 종류(출금/입금/잔액조회/이체)에 따라 처리한다. CashDispenserTask는 TransactionTask로부터 받은 출금 요청에 따라 현금 재고 및 계좌 차감을 수행한다. LogTask는 TransactionTask로부터 입금/잔액조회/이체 내역과 CashDispenserTask로부터 출금 내역을 받아 거래 내역을 log.txt에 저장한다. ResetTask는 1분 주기로 모든 계좌의 일일 출금량을 초기화한다.

우선 순위로는 ResetTask > CashDispenser > TransactionTask > AuthTask > CardReaderTask > LogTask로 잡았다. 이유로는 CashDispenser는 출금 응답성이 가장 중요하며, 사용자가 가장 민감하게 체감하는 단계이기 때문이다. 그 다음으로 TransactionTask는 출금/입금/잔액조회/이체를 처리하므로 핵심 로직이어서 빠른 처리가 필요하기 때문이다. 그 다음으로 AuthTask는 거래 전 단계이지만, PIN 인증 수행(입력 대기 포함)하고 실시간성이 중간이기 때문이다. CardReaderTask는 상대적으로 실시간성이 낮기 때문이다. LogTask는 거래 완료 후 수행되는 부수적 기록 작업으로 우선순위가 낮고, ResetTask는 주기적으로 백그라운드에서 수행되는 비실시간 관리 작업이지만 우선순위 스케줄링에 따라 1분마다 자동으로 초기화를 시키려고 최우선순위를 부여했다.

<Semaphore>

사용한 Semaphore는 총 2개로, AccountSem, CashBoxSem이 있다. 먼저 AccountSem은 계좌 배열(accounts[])과 관련된 모든 접근을 상호 배제 방식으로 보호한다. AccountSem은 TransactionTask에서 입금, 출금, 이체 등의 계좌 잔액 업데이트 할 때, CashDispenserTask에서 출금 후

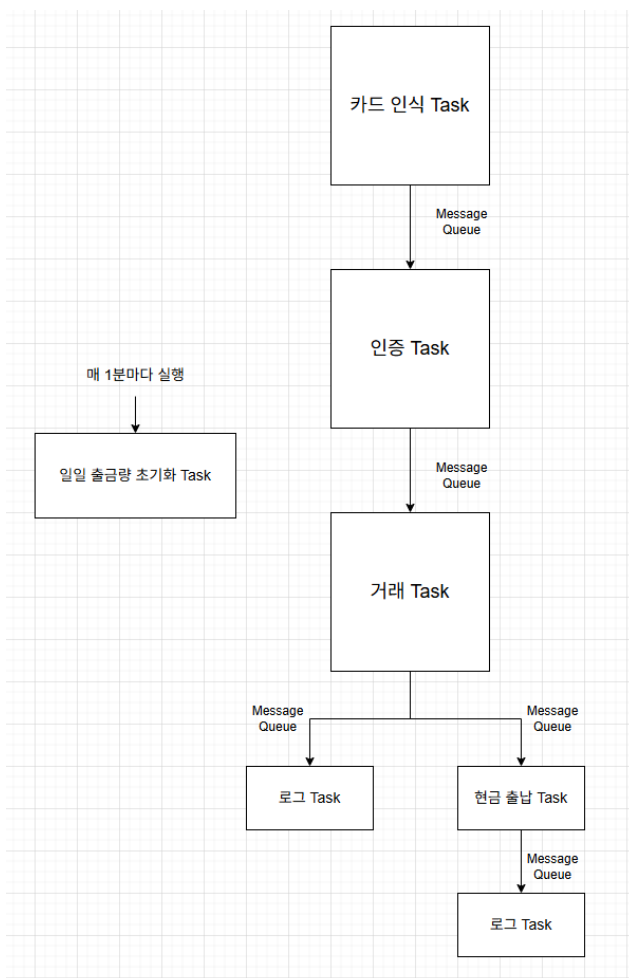
계좌 정보 업데이트 할 때, ResetTask에서 일일 출금량 초기화 할 때 사용한다. 이는 여러 Task가 동시에 계좌 정보를 수정하는 것을 방지하여 데이터의 정합성 유지 및 race condition을 방지한다. CashBoxSem은 ATM 내부의 현금 재고 변수인 cash_stock에 접근 시 동기화를 위해 사용된다. CashBoxSem은 CashDispenserTask에서 현금 재고 감소 처리 및 출금 가능 여부를 확인할 때 사용한다. 이는 현금 출납 처리의 일관성 유지 및 다중 Task가 동시에 현금 재고를 수정하는 경우를 방지한다.

<Message Queue>

사용한 Message Queue는 총 4개로, CardToAuthQ, AuthToTransQ, TransToCashQ, LogQ가 있다. CardToAuthQ는 CardReaderTask에서 읽은 카드 정보를 AuthTask로 전달한다. AuthToTransQ는 AuthTask에서 인증된 계좌정보를 TransactionTask로 전달한다. TransToCashQ는 TransactionTask에서 출금 요청 정보를 CashDispenserTask로 전달한다. LogQ는 거래 완료 정보를 LogTask로 전달하여 거래 기록을 저장한다.

2. 본문

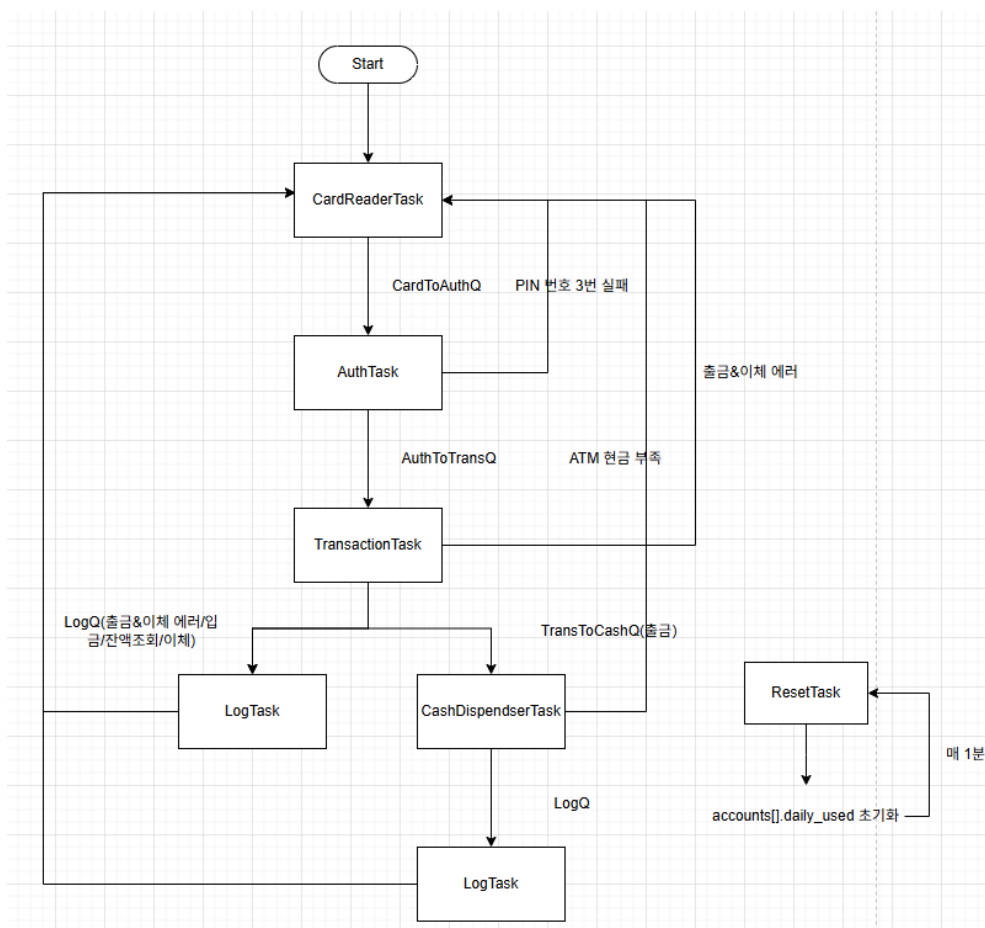
<프로젝트 개요>



위 사진은 프로젝트 전반을 설명하는 그림이다. ATM의 각 기능을 Task로 나누어 스케줄링 기능을 통해

동시에 실행된다. 위 사진을 보면 알다시피, 카드인식 → 인증 → 거래 → 출금/입금/잔액조회/이체 → 로그 기록으로 실행된다. Task 간에는 OSQCreate로 생성된 메시지 큐를 통해 정보가 전달된다. 예를 들어, CardInfo, AuthResult, TransactionInfo 등 구조체를 사용하여 데이터를 전달한다. 또한 공용 데이터(계좌 정보, 현금 재고 등)는 OSSemCreate로 생성된 세마포어를 통해 보호되며, race condition을 방지한다. 거래 정보는 로그 파일인 log.txt에 저장되며, 계좌 데이터인 accounts.txt는 시뮬레이터가 시작하면 로드되고, 종료 전에 새로 저장된다. 작성한 kernel function인 OSResetDailyLimits()은 1분 단위(실제로는 하루 단위)로 일일 누적 출금량을 초기화하며, ResetTask에서 호출된다. 이 프로젝트는 RTOS 구조와 개념을 실제로 적용해봄으로써, ATM 동작의 전체 흐름을 간접적으로 이해할 수 있었다.

<프로젝트의 구조>



위 사진은 프로젝트의 구조를 Flow chat로 간단히 표현한 것이다.

<각 task의 정의>

1. CardReaderTask: 사용자의 카드가 삽입되었을 때, 카드 정보를 읽고 유효성(만료 여부)을 검사한 후에 AuthTask로 전달한다. 우선순위 6을 부여한다.
2. AuthTask: 카드에 매핑된 계좌를 찾아 사용자 PIN 인증을 수행하고, 성공 시 인증 결과를 TransactionTask로 전달한다. 우선순위 5를 부여한다.

3. TransactionTask: 인증된 계좌에 대해 사용자의 거래 요청(출금/입금/잔액조회/이체)을 처리하고, 거래 정보 생성 후 해당 Task(CashDispenserTask 또는 LogTask)로 전달한다. 우선순위 4를 부여한다.
4. CashDispenserTask: TransactionTask로부터 출금 요청을 전달받고, 현금 출납 가능 여부를 확인한 후에, 실제 계좌 차감 및 일일 출금량을 반영하고 거래 정보를 LogTask로 전달한다. 우선순위 3을 부여한다.
5. LogTask: 각 거래 결과(TransactionInfo)를 log.txt에 기록한다. 우선순위 10을 부여한다.
6. ResetTask: 1분 간격으로 모든 계좌의 일일 출금 사용량(daily_used)을 커널 함수를 호출하여 0으로 초기화한다. 우선순위 1을 부여한다.

우선순위: ResetTask > CashDispenserTask > TransactionTask > AuthTask > CardReaderTask > LogTask > 먼저 CashDispenserTask가 가장 높은 우선순위를 가진 것은 출금은 사용자 대기 시간이 중요한 실시간 요소이므로 높은 우선순위를 부여했다. 그 다음으로 TransactionTask는 거래 처리 로직의 중심이고, 사용자 응답성과 처리 정확도를 위해 높은 우선순위를 부여했다. 그 다음으로 AuthTask는 인증은 거래 처리 이전에 반드시 수행되어야 하고, 인증 실패 시에도 빠르게 응답하기 위해 CardReaderTask보다 높은 우선순위를 부여했다. 그 다음으로 CardReaderTask는 시스템의 진입점으로, 계속 Ready 상태에 가까워 낮은 우선순위여도 자주 실행된다. 그 다음으로 LogTask는 각 Task 처리 후 기록이므로 낮은 우선순위를 부여했다. 그 다음으로 ResetTask는 백그라운드 작업이지만, 우선순위 스케줄링에 따라 1분마다 자동으로 초기화를 시키려고 최우선순위를 부여했다.

부여한 우선순위에 따르면 우선순위가 높은 순서대로 실행되어야 맞지만, 아무리 우선순위가 높아도 Block 상태에서는 실행되지 않는다. CardReaderTask, LogTask, ResetTask를 제외한 Task들은 OSQPend()로 메시지를 기다리며 Block 상태에 빠져 있다. 따라서, 누군가가 OSQPost()로 메시지를 보내기 전까지는 실행되지 않는다. 그러면 메시지를 받고, 우선 순위가 높은 Task가 Ready 상태가 되어 커널은 즉시 Context Switch를 발생시켜 그 Task를 실행한다.

<Task간 Semaphore와 Message Queue의 활용방안>

프로젝트에서 Semaphore는 공유 자원에 대한 상호배제를 보장하기 위한 동기화 도구로 활용했다. 먼저 사용한 Semaphore는 AccountSem과 CashBoxSem이 있다. AccountSem의 용도는 계좌 정보를 보호하기 위해 사용했고, 사용한 Task로는 TransactionTask, CashDispenserTask, ResetTask가 있다. 계좌 정보(accounts[])는 여러 Task에서 접근하므로, 동시에 접근할 경우 데이터 경합이 발생할 수 있다. 따라서 이를 방지하기 위해 OSSemPend()와 OSSemPost()로 보호한다. CashBoxSem은 ATM 기기의 현금 재고를 보호하기 위해 사용했고, 사용한 Task로는 CashDispenserTask가 있다. ATM 기기의 현금 재고(cash_stock)는 하나의 출금 Task만 접근해야 하므로 동기화가 필요하다. 여러 출금 Task가 동시에 접근할 경우 현금 부족 판단이 잘못될 수 있어서 Semaphore로 보호한다.

Message Queue는 Task 간 데이터 전달 및 흐름제어를 위해 활용되었다. 먼저 사용한 Message Queue는 CardToAuthQ, AuthToTransQ, TransToCashQ, LogQ가 있다. CardToAuthQ에서의 송신 Task는 CardReaderTask이고, 수신 Task는 AuthTask이다. 전달 목적은 카드 삽입 후 카드 정보를 AuthTask로 전달하기 위해서이다. AuthToTransQ에서의 송신 Task는 AuthTask이고, 수신 Task는 TransactionTask이다. 전달 목적은 인증 완료 후, 인증된 계좌 정보를 TransactionTask로 전달하기 위해서이다. TransToCashQ에서의 송신 Task는 TransactionTask이고, 수신 Task는 CashDispenserTask이다. 전달 목적은 출금 요청 시, 거래 정보를 전달하고 처리하기 위해서이다. LogQ에서의 송신 Task는 TransactionTask와 CashDispenserTask이고, 수신 Task는 LogTask이다. 전달 목적은 모든 거래 기록 및 실패 로그를 LogTask로 전달하여 log.txt에 기록하기 위해서이다.

<제안서 대비 변경사항>

주제가 바뀌어서 이전 제안서의 내용은 의미가 없다.

이번 프로젝트에 RTOS가 필요한 이유: Semaphore를 활용한 자원 공유 및 보호, Message Queue를 활용한 Task 간 통신, 우선순위 기반 스케줄링, 멀티태스킹

<동작 예시>

```
012345 배정우 50000 100000 0 1234
123456 박성준 100000 100000 0 2345
234567 나원호 150000 100000 0 3456
345678 권혁준 200000 100000 0 4567
456789 현예림 250000 200000 0 5678
567890 김영빈 300000 200000 0 6789
678900 최동찬 100000 100000 0 7890
```

위 사진은 초기의 account.txt의 모습이다.

```
=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 22222222)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 3456
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
1. 출금 2. 입금 3. 잔액조회 4. 이체
[거래처리] 선택: 1
[거래처리] 출금을 선택하셨습니다.
[거래처리] 출금액을 입력하세요: 500
[현금출납] 현금 준비 중... 500원 출금 완료
[시스템] 거래가 완료되었습니다. 카드를 회수하세요.

=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 00000000)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 1234
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
1. 출금 2. 입금 3. 잔액조회 4. 이체
[거래처리] 선택: 2
[거래처리] 입금을 선택하셨습니다.
[거래처리] 입금액을 입력하세요: 500
[거래처리] 500원 입금 완료. 현재 잔액: 50500원
[시스템] 거래가 완료되었습니다. 카드를 회수하세요.
```

```
=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 22222222)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 3456
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
1. 출금 2. 입금 3. 잔액조회 4. 이체
[거래처리] 선택: 3
[거래처리] 잔액조회를 선택하셨습니다.
[거래처리] 현재 잔액: 149500원
[시스템] 거래가 완료되었습니다. 카드를 회수하세요.
```

위의 사진들과 같이 출금, 입금, 잔액조회를 진행한 후에 accounts.txt의 모습과 log.txt의 모습을 보면

아래의 사진과 같다.

```
[거래기록]
계좌번호 : 234567
거래종류 : 출금
거래금액 : 500원
거래시간 : 2025-06-01 22:20
상태 : 성공
-----
[거래기록]
계좌번호 : 012345
거래종류 : 입금
거래금액 : 500원
거래시간 : 2025-06-01 22:20
상태 : 성공
-----
[거래기록]
계좌번호 : 234567
거래종류 : 잔액조회
거래금액 : 0원
거래시간 : 2025-06-01 22:20
상태 : 성공
-----
012345 배정우 50500 100000 0 1234
123456 박성준 100000 100000 0 2345
234567 나원호 149500 100000 0 3456
345678 권혁준 200000 100000 0 4567
456789 현예림 250000 200000 0 5678
567890 김영빈 300000 200000 0 6789
678900 최동찬 100000 100000 0 7890

=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 33333333)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 4567
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
1. 출금 2. 입금 3. 잔액조회 4. 이체
[거래처리] 선택: 4
[거래처리] 이체를 선택하셨습니다.
[거래처리] 이체 대상 계좌번호 입력: [Kernel] 일일 출금 사용량 초기화 완료
[Kernel] 일일 출금 사용량 초기화 완료
[Kernel] 일일 출금 사용량 초기화 완료
```

위 사진들은 입금, 출금, 잔액조회를 했을 때 log.txt에 기록되는 내용과 각 계좌에 출금으로 인해 계좌 잔액 차감과 입금으로 인한 계좌 잔액 증감을 볼 수 있다. 또한 커널 함수로 인해 매 1분마다 일일 출금 사용량이 초기화되어서 각 계좌들의 일일 출금량이 모두 0으로 초기화된 것을 확인할 수 있다. 아래의 사진은 커널 함수로 인해 초기화되기 전의 모습이다.

```
[거래기록]
계좌번호 : 234567
거래종류 : 이체
거래금액 : 50000원
거래시간 : 2025-06-01 22:31
상태 : 성공
-----
[거래기록]
계좌번호 : 567890
거래종류 : 출금
거래금액 : 50000원
거래시간 : 2025-06-01 22:31
상태 : 성공
-----
012345 배정우 100500 100000 0 1234
123456 박성준 100000 100000 0 2345
234567 나원호 99500 100000 0 3456
345678 권혁준 200000 100000 0 4567
456789 현예림 250000 200000 0 5678
567890 김영빈 250000 200000 50000 6789
678900 최동찬 100000 100000 0 7890
```

```

=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 22222222)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 3456
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
          1. 출금  2. 입금  3. 잔액조회  4. 이체
[거래처리] 선택: 4
[거래처리] 이체를 선택하셨습니다.
[거래처리] 이체 대상 계좌번호 입력: 012345
[거래처리] 이체액을 입력하세요: 50000
[거래처리] 50000원 이체 완료. 현재 잔액: 99500원
[시스템] 거래가 완료되었습니다. 카드를 회수하세요.

=== ATM 기계 시뮬레이션 ===
[카드인식] 카드가 삽입되었습니다. (카드 번호: 55555555)
[카드인식] 카드 정보 읽는 중... 완료
[PIN인증] PIN 번호 4자리를 입력하세요: 6789
[PIN인증] 인증 중... 성공
[거래처리] 거래 메뉴를 선택하세요
          1. 출금  2. 입금  3. 잔액조회  4. 이체
[거래처리] 선택: 1
[거래처리] 출금을 선택하셨습니다.
[거래처리] 출금액을 입력하세요: 50000
[현금출납] 현금 준비 중... 50000원 출금 완료
[시스템] 거래가 완료되었습니다. 카드를 회수하세요.

```

위 사진들은 커널 함수로 인해 일일 출금량이 초기화되기 전에 출금을 하고 이체를 했을 때의 accounts.txt와 log.txt의 모습이다. 일일 출금량에 50000이 추가된 것을 확인할 수 있고 이체를 통해 계좌의 잔액이 5만원 증감과 차감이 이루어진 것을 확인할 수 있다.



광운대학교
KwangWoon University